Desafio — Agente de Pagamentos na AWS

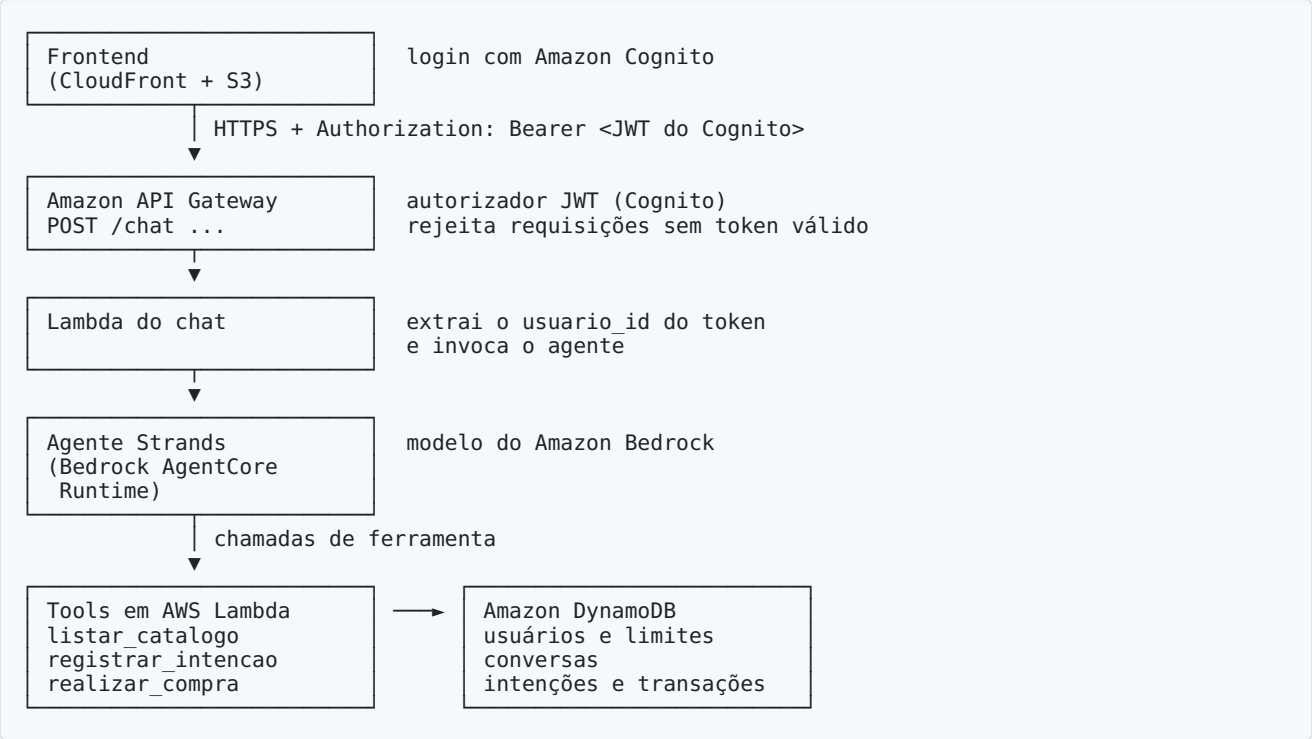
Trilha: Pagamentos Agênticos · Sprint 2 · Trabalho individual

Levar para a AWS o desafio final da Fase 1: o chatbot com login que conversa com um agente de IA e realiza compras simuladas por meio de três ferramentas. A aplicação, que hoje roda localmente com Ollama, passa a rodar na nuvem com um modelo do Amazon Bedrock, o agente implantado no Amazon Bedrock AgentCore e toda a infraestrutura definida como código.

As **regras de negócio não mudam**: catálogo, intenção de compra, compra com cartão ou Pix, limite por usuário e validação da intenção continuam valendo exatamente como na Fase 1. O que muda é onde e como cada peça roda.

O desafio é **individual**. Todos os fellows trabalham no mesmo repositório compartilhado, cada um na sua própria pasta.

Arquitetura esperada



O desenho acima é a referência. Variações são aceitas se estiverem justificadas na documentação, por exemplo expor as Lambdas ao agente pelo AgentCore Gateway como ferramentas MCP. A comunicação entre o frontend e o backend, porém, deve passar obrigatoriamente pelo API Gateway.

Stack

Camada	Obrigatório	Recomendado
Frontend	Amazon S3 (bucket privado) servido pelo Amazon CloudFront	SPA estática (React com Vite, ou Next.js com exportação estática); qualquer framework que gere arquivos estáticos é aceito
Autenticação	Amazon Cognito	Biblioteca `aws-amplify` com o componente `Authenticator` (`@aws-amplify/ui-react`), apontando para o user pool criado pelo seu IaC. Não é preciso usar o Amplify Hosting
API do webchat	Amazon API Gateway com autorizador JWT do Cognito	HTTP API com JWT authorizer
Agente	Strands Agents no Bedrock AgentCore Runtime	Python ou TypeScript
Região	**us-east-1** (Norte da Virgínia) para todos os recursos	—
Modelo	Um modelo do Amazon Bedrock	**Claude Haiku**, da Anthropic
Ferramentas	AWS Lambda (uma por tool, ou uma com roteamento interno)	—
Banco de dados	Amazon DynamoDB	—
Infraestrutura	AWS CDK ou AWS CloudFormation ou Terraform	—

> **Todos os recursos devem ser criados na região us-east-1.** Concentrar tudo em uma região evita erros difíceis de diagnosticar, como um recurso que não encontra o outro, e facilita a correção e a limpeza ao final.

Requisitos

1. Frontend e autenticação

- Aplicação estática hospedada em um bucket **S3 privado** e servida pelo **CloudFront**, acessível pela URL pública da distribuição (`https://<id>.cloudfront.net`).
- O bucket não pode ser público nem usar website hosting do S3. O acesso deve ser restrito ao CloudFront por **Origin Access Control (OAC)**.
- A configuração do frontend (URL da API, IDs do user pool e do app client) é gerada pelo IaC no deploy, por exemplo como um `config.json` enviado ao bucket. Nada disso fica fixo no código.

- Login com Amazon Cognito. O chat não pode ser acessado sem autenticação.
- A identidade do usuário deve ser validada **no backend**, a cada requisição. Esconder o chat na tela não é controle de acesso.

Cadastro somente por convite

Para evitar que terceiros criem contas e consumam o modelo enquanto a aplicação está no ar aguardando correção, **o auto cadastro deve estar desabilitado**:

- O user pool do Cognito deve permitir apenas a criação de usuários por um administrador (`AllowAdminCreateUserOnly`). Com isso, chamadas de cadastro feitas diretamente à API do Cognito também são recusadas, e não só escondidas na tela.
- Novos usuários são criados pelo administrador, pelo console do Cognito ou pelo comando `aws cognito-idp admin-create-user`.
- O Cognito envia ao usuário uma **mensagem de convite** (*invitation message*) por e-mail, com o nome de usuário e uma **senha temporária**. No primeiro login, o usuário é obrigado a definir uma nova senha.
- A tela de login não deve exibir a opção de cadastro. No componente `Authenticator`, use a propriedade `hideSignUp`.

A configuração deve estar no código de infraestrutura:

Ferramenta	Configuração
AWS CDK	`selfSignUpEnabled: false` no `UserPool`, e `userInvitation` para personalizar o e-mail de convite
CloudFormation	`AdminCreateUserConfig.AllowAdminCreateUserOnly: true` e `InviteMessageTemplate`
Terraform	no `aws_cognito_user_pool`, bloco `admin_create_user_config` com `allow_admin_create_user_only = true` e `invite_message_template`

2. API Gateway do webchat

- Os endpoints que o frontend usa para o chat (por exemplo, `POST /chat` para enviar mensagens e `GET /conversas` para carregar o histórico) devem ser expostos pelo **Amazon API Gateway**.
- A autenticação deve usar o **JWT do Cognito**, validado por um autorizador no próprio API Gateway. O frontend envia o token no header `Authorization: Bearer <token>`.
- Requisições sem token, com token expirado ou inválido devem ser rejeitadas pelo API Gateway com `401`, sem chegar à Lambda.
- A Lambda do chat obtém o `usuario_id` das **claims do token** (o `sub` do Cognito), nunca do corpo da requisição.
- CORS configurado para aceitar apenas a origem do app, ou seja, o domínio da distribuição do CloudFront.
- O API Gateway deve ser ou Stream ou Websocket para criar uma melhor experiência de usuário.

> **Por que JWT e não API key?** Uma API key identifica a aplicação, não o usuário: todos os usuários enviariam a mesma chave, e o backend não saberia quem está comprando. Além disso, ela ficaria exposta no JavaScript do navegador. A própria AWS recomenda não usar API keys como mecanismo de autenticação ou autorização. Elas servem para planos de uso e limites de requisições.

> **Atenção ao tempo de resposta:** o API Gateway tem um tempo máximo de integração. Uma resposta do agente que encadeia várias ferramentas pode se aproximar desse limite. Consulte o limite atual na documentação do API Gateway e escolha um modelo com boa latência.

3. Agente

- Agente construído com Strands Agents e implantado no Bedrock AgentCore Runtime.
- Usar um modelo do Amazon Bedrock no lugar do Ollama.
- O histórico completo da conversa continua sendo enviado ao agente a cada turno.

4. Ferramentas em Lambda

- As três ferramentas da Fase 1 (`listar_catalogo`, `registrar_intencao` e `realizar_compra`) passam a ser executadas em funções AWS Lambda.
- Os contratos de argumentos e retorno são os mesmos da Fase 1, incluindo os códigos de erro (`INTENCAO_INVALIDA`, `INTENCAO_EXPIRADA`, `INTENCAO_JA_PAGA`, `LIMITE_EXCEDIDO` e `METODO_INVALIDO`).

5. Persistência em DynamoDB

Na Fase 1 o estado ficava em memória. Na nuvem isso não funciona: Lambdas e containers são efêmeros e podem rodar em várias instâncias ao mesmo tempo. Tudo que precisa sobreviver entre requisições vai para o DynamoDB:

- **Usuários:** identificador do Cognito e limite de gasto disponível.
- **Conversas:** histórico de mensagens de cada usuário.
- **Intenções e transações:** intenções registradas, status e compras realizadas.

A modelagem das tabelas é livre e deve estar explicada na documentação.

6. Regras de segurança (as mesmas da Fase 1)

- O **`usuario_id` vem da sessão autenticada do Cognito**, nunca de um argumento que o modelo preenche.
- O **valor da compra vem da intenção registrada** no banco, nunca de um parâmetro enviado pelo modelo.
- `realizar_compra` só aceita intenções criadas pelo backend, pertencentes ao usuário e ainda não pagas.

- A atualização do limite e o pagamento da intenção não podem gerar duplicidade se a mesma chamada chegar duas vezes. Dica: escritas condicionais do DynamoDB.

7. Infraestrutura como código e deploy

- Toda a aplicação deve ser criada por **AWS CDK, CloudFormation ou Terraform**, com o código na pasta `infra/`: frontend (bucket S3, distribuição CloudFront, OAC e upload dos arquivos), Cognito, API Gateway, tabelas, Lambdas, agente, roles e permissões.
- **Não há CI/CD.** O deploy é feito rodando o script IaC da sua pasta. Um único comando, documentado no seu README, deve criar o ambiente completo, incluindo o build e o upload do frontend e a invalidação do cache do CloudFront.
- Como várias pessoas podem fazer deploy na mesma conta, use o **seu nome como prefixo** da stack e dos recursos para evitar colisões.
- Criar recursos manualmente pelo console só é aceito para o que não tiver alternativa via código, e deve estar documentado.

8. Menor privilégio no IAM

Cada componente recebe apenas as permissões de que precisa:

Quem	Precisa de	Não deve ter
Lambda do chat (atrás do API Gateway)	Invocar o runtime do agente e ler/gravar a tabela de conversas	Acesso às tabelas de intenções e transações ou ao Bedrock
Role do AgentCore Runtime	Invocar o modelo escolhido, chamar as Lambdas das tools, gravar logs	Acesso direto às tabelas
Cada Lambda de tool	Ler e escrever apenas nas tabelas que usa	Permissões de administrador ou `*`
Bucket do frontend	Leitura (`s3:GetObject`) apenas pela sua distribuição do CloudFront, via OAC	Acesso público ou de qualquer outra origem

Não é aceito `"Action": "*"` com `"Resource": "*"`, nem políticas gerenciadas amplas como `AdministratorAccess`. As políticas de cada role devem estar no código de infraestrutura.

9. Boas práticas de programação

- Nenhuma credencial, chave de acesso ou segredo no código ou no histórico do Git. Em um repositório compartilhado, um segredo publicado fica visível para todos.
- `.gitignore` configurado (`.env` , `node_modules` , `cdk.out` , `.terraform` , `*.tfstate` e afins).
- Todo o seu trabalho fica em `fellows/<seu-nome>/` , organizado em pastas por responsabilidade (`frontend/` , `api/` , `agent/` , `tools/` , `infra/`). Não altere a pasta de outro fellow.
- Commits pequenos e descritivos, em uma branch sua, seguindo o fluxo de branches e pull requests das sprints anteriores.
- Tratamento de erros: falhas de ferramenta devem chegar ao usuário como mensagens compreensíveis, não como stack traces.

Critérios de conclusão

- ☐ Frontend publicado em S3 + CloudFront, com URL pública e bucket privado (OAC).
- ☐ Login com Cognito funcionando; chat inacessível sem autenticação.
- ☐ Auto cadastro desabilitado; usuários criados apenas por convite do administrador.
- ☐ Endpoints do webchat expostos pelo API Gateway, com autorizador JWT do Cognito.
- ☐ Requisição sem token ou com token inválido rejeitada com `401` pelo API Gateway.
- ☐ Agente Strands rodando no Bedrock AgentCore Runtime, com modelo do Bedrock.
- ☐ As três ferramentas executando em Lambda.
- ☐ Usuários, conversas e intenções persistidos no DynamoDB.
- ☐ Roles com menor privilégio, sem permissões curinga.
- ☐ Ambiente completo criado rodando o script IaC, sem passos manuais não documentados.
- ☐ Pull request com a sua pasta, sem credenciais, com a documentação descrita no README.

Entrega

Um **pull request no repositório compartilhado** com a pasta `fellows/<seu-nome>/`, contendo o código e um `README.md` com:

1. **Diagrama da arquitetura** como foi implementada.
2. **Instruções de deploy** passo a passo, incluindo pré-requisitos e o comando do script IaC.
3. **Modelo do Bedrock usado** (e o motivo, caso não seja o Claude Haiku recomendado).
4. **Modelagem do DynamoDB:** tabelas, chaves e o que cada uma guarda.
5. **Resumo das permissões IAM** de cada role.
6. **Evidências** (prints ou vídeo curto) de:
    - login;
    - chamada à API sem token sendo rejeitada com `401` (por exemplo, com `curl` ou Postman);
    - compra bem-sucedida com cartão e com Pix;
    - tentativa bloqueada por limite excedido;
    - tentativa com `intencao_id` inválido sendo recusada.
7. **Instruções para remover** todos os recursos criados.

> **Antes de abrir o pull request**, revise o histórico dos seus commits. Uma chave de acesso publicada no GitHub é encontrada por bots em minutos. Se algo vazou, desative a chave no IAM imediatamente; apagar o commit não basta.

A aplicação deve **permanecer no ar até a correção**. Antes de entregar, crie pelo console do Cognito um usuário com o e-mail do instrutor responsável pela correção. Ele receberá o convite com a senha temporária e conseguirá acessar a aplicação.

Custos e limpeza

- Configure um alerta no **AWS Budgets** antes do primeiro deploy.
- Use o modelo recomendado, o Claude Haiku, que tem custo baixo e bom desempenho em tool calling. Antes de começar, confirme no console do Bedrock que o acesso ao modelo está habilitado na conta, em us-east-1.
- **Depois que o desafio for corrigido**, remova todos os recursos para evitar cobranças:
  - a stack (`cdk destroy`, exclusão da stack no CloudFormation ou `terraform destroy`);
  - o bucket do frontend e a distribuição do CloudFront, caso não tenham sido removidos com a stack (um bucket com objetos não é excluído; esvazie-o antes ou configure a remoção automática no IaC);
  - o runtime do agente no AgentCore e as imagens no Amazon ECR, se tiverem sido criados fora da stack;
  - tabelas do DynamoDB e grupos de logs no CloudWatch que tenham ficado para trás.

Confira no console que nada restou. Recursos criados fora da stack não são removidos pelo destroy.

Extras (opcionais)

- Expor as Lambdas ao agente como ferramentas MCP pelo **AgentCore Gateway**.
- Usar o **AgentCore Memory** para lembrar preferências do usuário entre sessões.
- Aplicar **Guardrails do Bedrock** para impedir que o agente exponha dados sensíveis ou fale de assuntos fora do escopo.
- Configurar observabilidade com traces do agente no **CloudWatch**.

Material de apoio

- CloudFront — Restringir o acesso a uma origem S3 (OAC)
- AWS CDK — Upload de arquivos para o S3 (`aws-s3-deployment`)
- Amplify UI — Componente Authenticator (React)
- Strands — Deploy no Bedrock AgentCore (Python)
- Strands — Deploy no Bedrock AgentCore (TypeScript)
- AWS — Build and deploy an AI agent with Bedrock, Strands and AgentCore
- AWS CDK — Serverless Hello World
- API Gateway — Autorizadores JWT em HTTP APIs
- Cognito — Políticas de criação de usuários (desabilitar auto cadastro)
- Terraform — `aws_cognito_user_pool`
- Exemplo de referência — aws-samples/sample-strands-agent-with-agentcore